محرمانه · فقط برای مدرس

📘 راهنمای مدرس — چالش «بساز و منتشر کن»

⚠️ **محرمانه.** این فایل و کل پوشه‌ی `instructor/` (تصحیح‌گرها، راه‌حل‌های مرجع، بانک سؤال دفاع) را **به دانشجوها ندهید.** دانشجوها چک‌لیست آماده‌سازی را یک هفته قبل، و فقط `README.md` و `PLAN.md` نسخه‌ی خودشان را روز آزمون می‌گیرند.

۱. خلاصه‌ی آزمون در یک نگاه

مورد	تصمیم
شکل آزمون	یک آزمون ترکیبی (Go + DevOps) با **دو نسخه‌ی هم‌سطح**: **A = MiniBank** و **B = SnapLink**
مخاطب	دانشجوهایی که هر ۲۰ جلسه‌ی دوره‌ی Go و هر ۳۲ جلسه‌ی بوت‌کمپ DevOps را تمام کرده‌اند
زمان	۱۰ دقیقه مطالعه + ۳ ساعت کار، توقف قطعی
شیوه	انفرادی، از پوشه‌ی خالی
ابزار	همه‌چیز آزاد است، حتی هوش مصنوعی؛ کمک گرفتن از انسان ممنوع است
امتیاز	۸۰ ساخت + ۲۰ دفاع حضوری = ۱۰۰، به‌علاوه‌ی حداکثر ۱۵ امتیاز اضافه
قبولی	۶۰ (نمره‌ی نهایی به نزدیک‌ترین عدد صحیح گرد می‌شود؛ امتیاز اضافه هم حساب است)
رتبه‌ها	۹۰+ افسانه · ۸۰+ حرفه‌ای · ۷۰+ سازنده · ۶۰+ قبول · زیر ۶۰ آزمون مجدد
تصحیح	تصحیح‌گر خودکار (Python) + چک‌لیست دستی + دفاع ۷ تا ۱۲ دقیقه‌ای بدون AI
قانون کلیدی	**«توضیح بده یا از دست بده»**: اگر دانشجو بعد از دو سؤال ساده‌تر نتواند یک مورد امتیازی را توضیح بدهد، امتیاز همان مورد صفر می‌شود (ساخت یا اضافه)
محیط	لپ‌تاپ خود دانشجو با Docker و kind (یا minikube)؛ ویندوز با WSL2
تحویل	ریپازیتوری GitHub (عمومی، یا خصوصی با دعوت از حساب GitHub شما)
زبان	همه‌ی اسناد به فارسی؛ کد، دستورها و اصطلاح‌های فنی انگلیسی

نقشه‌ی فایل‌ها

```
student-challenges/
├── START-HERE.md                          # راهنمای توزیع فایل‌ها
├── PDF/                                   # PDFهای دانشجو (برای چاپ یا ارسال)
├── 00-setup-checklist/                    # چک‌لیست آماده‌سازی (یک هفته قبل بفرستید)
├── version-A-minibank/                    # برای دانشجوهای نسخه‌ی A: README.md و PLAN.md
├── version-B-snaplink/                    # برای دانشجوهای نسخه‌ی B: README.md و PLAN.md
└── instructor/                            # فقط برای شما
    ├── Instructor-Guide-CONFIDENTIAL.pdf  # همین راهنما PDF
    ├── INSTRUCTOR-GUIDE.md                # همین فایل
    ├── graders/                           # grade_minibank.py و grade_snaplink.py
    ├── reference/                         # راه‌حل مرجع کامل هر دو نسخه
    └── tools/                             # اسکریپت ساخت دوباره‌ی PDFها
```

---

۲. منوی ایده‌ها

ایده‌هایی که انتخاب شدند

نسخه	پروژه	نقطه‌ی قوت	تله‌ی اصلی هم‌روندی
A	**MiniBank**: دفتر کل بانکی روی PostgreSQL	تراکنش SQL، اعتبارسنجی پول، بیشترین پوشش مفاهیم Go	«بخوان، بررسی کن، بنویس» بدون قفل در انتقال، که پول را گم می‌کند
B	**SnapLink**: کوتاه‌کننده‌ی لینک روی Redis	ریدایرکت، امنیت URL، دستورهای اتمیک Redis	«GET بعد SET» برای کلیک، که کلیک‌ها را گم می‌کند

هر دو نسخه **دقیقاً همان ۹ مرحله و همان امتیازها** را دارند و فقط اپلیکیشن فرق می‌کند. پس می‌توانید آن‌ها را **یکی‌درمیان به صندلی‌ها** بدهید (همسایه‌ها نسخه‌ی متفاوت داشته باشند) یا یکی را برای آزمون مجدد نگه دارید.

ایده‌های دیگر برای آزمون‌های بعدی

ایده	خلاصه	مناسب برای
LogLens	ابزار CLI هم‌روند برای تحلیل لاگ (worker pool، گزارش top-N، خروجی JSON)	آزمون فقط Go، تمرکز روی هم‌روندی و فایل
KVStore	سرور key-value با TTL، goroutine پاک‌کننده و snapshot روی دیسک	آزمون فقط Go، هم‌سطح MiniBank
QuizBox	آزمونک ترمینالی زمان‌دار با `select` و `time.After` و فایل JSON سؤال‌ها	دانشجوهای مبتدی (جلسات ۱ تا ۱۵)
StatusPulse	پایشگر uptime که لیست URLها را از ConfigMap می‌خواند و `/metrics` دارد	تمرکز روی Prometheus؛ به اینترنت پایدار نیاز دارد
NoteDrop	API یادداشت روی PostgreSQL با migration، و در Kubernetes با StatefulSet و PVC	تمرکز روی داده و ذخیره‌سازی پایدار
OrderFlow	دو میکروسرویس (سفارش و انبار) پشت Nginx Ingress	دانشجوهای قوی یا آزمون ۶ ساعته

ایده	خلاصه	مناسب برای
JobRunner	صف کار روی Redis با worker جدا و CronJob در Kubernetes	تمرکز روی الگوهای میکروسرویس (جلسات ۱۸ تا ۲۰)

ایده‌های قالب برای جذاب‌تر کردن:

- **«حادثه‌ی زنده»** (اختیاری و بدون امتیاز؛ مثلاً در دفاع به‌جای یکی از خرابی‌های D3): اعلام کنید «رمز datastore عوض شد». در A رمز داخل خود Postgres هم باید عوض شود (`kubectl -n minibank exec deploy/postgres -- psql -U minibank -d minibank -c "ALTER USER minibank WITH PASSWORD '<new>'"`)، بعد هر دو کلید `POSTGRES_PASSWORD` و `DATABASE_URL` در Secret به‌روز شوند و `kubectl -n minibank rollout restart deploy/minibank` زده شود. در B، Secret رمز Redis به‌روز می‌شود و هر دو Deployment (`redis` و `snaplink`) restart می‌شوند. اگر بخواهید امتیاز داشته باشد، قبل از آزمون در README هر دو نسخه اعلام کنید و سقف ۱۵ امتیاز اضافه را رعایت کنید.
- **جدول رده‌بندی (leaderboard):** نمره‌ی خودکار هر نفر را روی تخته بنویسید.
- **نسخه‌ی C برای آزمون مجدد:** یکی از ایده‌های بالا را با همین قالب ۹ مرحله‌ای بسازید.

---

۳. تصمیم‌ها و فرض‌ها (سابقه‌ی روش Q)

این آزمون با روش Q طراحی شد: **تحلیل، بعد سؤال، بعد توافق، بعد ساخت**. این‌ها جواب‌های شما بودند:

سؤال	جواب شما
ساختار آزمون	یک آزمون ترکیبی
پیشرفت دانشجوها	هر ۳۲ جلسه‌ی بوت‌کمپ (و ۲۰ جلسه‌ی Go) تمام شده
اپ Go	MiniBank
اپ DevOps	SnapLink
استفاده از دو اپ	نسخه‌ی A و نسخه‌ی B، هم‌سطح
زبان	ابتدا انگلیسی و فارسی، بعد «همه‌چیز فارسی»؛ پس همه‌ی اسناد فارسی شدند
منابع مجاز	همه‌چیز، حتی هوش مصنوعی
تصحیح	تصحیح‌گر خودکار + دفاع
وزن دفاع	۲۰ از ۱۰۰
نمره‌ی قبولی	۶۰
محیط	لپ‌تاپ خود دانشجو با کلاستر محلی
شیوه	انفرادی

فرض‌هایی که بدون پرسیدن گرفته شد (هر کدام را خواستید عوض کنید):

۱. **MiniBank روی PostgreSQL است** (نه فایل JSON)، چون Kubernetes دو رپلیکا دارد و فایل محلی با دو رپلیکا داده را دو تکه می‌کند. این‌طوری A و B ساختار یکسانی دارند: اپ + یک datastore.

۲. **هر دو نسخه حالت** `STORE=memory` **دارند** تا دانشجو بتواند قبل از راه‌اندازی دیتابیس کار کند، تست‌ها بدون دیتابیس اجرا شوند، و CI ساده بماند. **ولی امتیاز هم‌روندی (و B4 در نسخه‌ی A) فقط روی datastore واقعی حساب می‌شود.**

۳. **فریم‌ورک وب ممنوع است** (فقط `net/http`) تا دانش Go سنجیده شود؛ فقط درایور datastore و (برای امتیاز اضافه) کلاینت Prometheus مجاز است.

۴. **تحویل از طریق GitHub** است، چون مرحله‌ی CI به GitHub Actions نیاز دارد.

۵. **تصحیح روی لپ‌تاپ خود دانشجو** انجام می‌شود، چون کلاستر و Compose همان‌جا در حال اجراست.

۶. **تصحیح‌گر خودکار به دانشجوها داده نمی‌شود.** دانشجو `smoke.sh` خودش را می‌نویسد؛ قرارداد API کاملاً در README آمده است.

۷. **هوش مصنوعی آزاد است، پس دفاع سنگین‌تر شد:** هم ۲۰ امتیاز دارد و هم قانون «توضیح بده یا از دست بده» اعمال می‌شود. دانشجو در README بخش «AI usage» می‌نویسد.

۸. **۱۰ دقیقه وقت مطالعه قبل از شروع ساعت** اضافه شد، چون README و PLAN طولانی‌اند و مرحله‌ی ۰ فقط ۱۰ دقیقه وقت دارد. اگر نمی‌خواهید، این خط را از README حذف کنید.

---

۴. برنامه‌ی برگزاری

یک هفته قبل

- ☐ فایل **چک‌لیست آماده‌سازی** (`00-setup-checklist`) را برای همه بفرستید. این فایل پروژه را لو نمی‌دهد.
- ☐ یک **mirror معتبر Docker Hub** برای کلاس انتخاب و امتحان کنید (مثلاً mirror ابرآروان یا هر mirrorی که خودتان به آن اعتماد دارید) و آدرسش را اعلام کنید. mirrorها زیاد عوض می‌شوند؛ حتماً همان هفته تستش کنید.
- ☐ روش تحویل آدرس ریپازیتوری را تعیین کنید (برگه‌ی حضور، یا فرمی که فقط شما جواب‌هایش را می‌بینید) و نام کاربری GitHub خودتان را آماده کنید.
- ☐ نسخه‌ی هر دانشجو (A یا B) را تعیین کنید: یکی‌درمیان روی صندلی‌ها.

یک روز قبل (اجرای آزمایشی)

از هر دانشجو بخواهید این‌ها را اجرا کند و خروجی را بفرستد:

```
go version
go env CGO_ENABLED
docker run --rm hello-world
kind get clusters
kubectl get nodes
python3 --version
ssh -T git@github.com
```

`go env CGO_ENABLED` باید `1` باشد (وگرنه `go test -race` در Linux و WSL کار نمی‌کند؛ راه حل: `sudo apt install -y build-essential`). نود kind باید `Ready` باشد.

روز آزمون

زمان	کار
۲۵ دقیقه قبل	چیدمان صندلی‌ها (A و B یکی‌درمیان)، اینترنت و برق را بررسی کنید
۱۵ دقیقه قبل	قوانین را بلند بخوانید (بخش قوانین README)، به‌خصوص «توضیح بده یا از دست بده»، «کمک انسانی ممنوع» و «فقط کامیت‌های pushشده»
۱۰ دقیقه قبل	فایل‌ها را بدهید: PDF برای خواندن و **حتماً** فایل `PLAN.md` نسخه‌ی خود دانشجو (از `version-A-minibank/` یا `version-B-snaplink/`) تا آن را در ریشه‌ی ریپازیتوری بگذارد. فقط خواندن؛ بدون کدنویسی و ترمینال
۰:۰۰	زمان را شروع کنید. ساعت شروع، ساعت ۰:۱۵ و ساعت ۳:۰۰ و نام کاربری GitHub خودتان را روی تخته بنویسید
در طول آزمون	فقط به سؤال‌های **روشن‌کننده درباره‌ی قرارداد** جواب بدهید و جواب را **برای همه** اعلام کنید. در اشکال‌زدایی کمک نکنید
۰:۱۵	آدرس همه‌ی ریپازیتوری‌ها را جمع کنید و برای هر کدام `git ls-remote <repo-url> refs/heads/main` بزنید و SHA را یادداشت کنید (مدرک push «تصمیم‌های من» در زمان)
۲:۳۰	هشدار «۳۰ دقیقه مانده»
۲:۵۰	هشدار «۱۰ دقیقه مانده: merge و push کنید»
۳:۰۰	«دست‌ها از کیبورد». برای هر ریپازیتوری دوباره `git ls-remote <repo-url> refs/heads/main` بزنید و SHA نهایی را ثبت کنید؛ فقط همین کامیت تصحیح می‌شود. بهتر است بخش HTTP تصحیح‌گر (بدون `--repo`) را همان موقع روی همه‌ی لپ‌تاپ‌ها اجرا کنید تا نتیجه‌ی API و هم‌روندی ثبت شود
بعد از آزمون	نوبت‌های تصحیح و دفاع، هر نفر حدود ۱۲ تا ۱۸ دقیقه (تصحیح‌گر ۲، بررسی دستی ۳، دفاع ۷ تا ۱۲)

> 💡 برای ۱۵ دانشجو، تصحیح و دفاع حدود ۳ تا ۴.۵ ساعت طول می‌کشد. با دو مدرس، یا با اجرای تصحیح‌گر برای همه درست بعد از ۳:۰۰ و برگزاری دفاع در روز بعد، زمان را کم کنید. اگر دفاع روز بعد است، از دانشجو بخواهید کلاستر را پاک نکند و چیزی را تغییر ندهد.

---

۵. چک‌لیست آماده‌سازی دانشجو

متن کامل در `00-setup-checklist` و خلاصه‌اش در انتهای README هر نسخه آمده است. نکته‌های مخصوص ایران و محیط:

مشکل	راه حل
Docker Hub آی‌پی ایران را مسدود می‌کند	تنظیم `registry-mirrors` در Docker Desktop یا `/etc/docker/daemon.json` ، یا VPN
`proxy.golang.org` کند است یا باز نمی‌شود	`go env -w GOPROXY=https://goproxy.io,direct` (فقط روی سیستم خود دانشجو اثر دارد)

مشکل	راه حل
داخل `docker build` ماژول‌ها دانلود نمی‌شوند	در مرحله‌ی ساخت Dockerfile، قبل از `RUN go mod download` : `ENV GOPROXY=https://goproxy.io,direct`
ایمیج نود kind (`kindest/node`) دانلود نمی‌شود	کلاستر را **روز قبل** بسازید و پاک نکنید
podهای postgres/redis در kind روی `ImagePullBackOff` می‌مانند	نود kind از mirror داکر استفاده نمی‌کند: `kind load docker-image postgres:16-alpine --name exam` (B: `redis:7-alpine`)
ایمیج‌های `gcr.io` و `registry.k8s.io` مسدودند	به‌جای distroless از `scratch` یا `alpine` استفاده شود. امتیاز اضافه‌ی Ingress (B2) به ایمیج‌های `registry.k8s.io` نیاز دارد؛ اگر دانلود نشد، دانشجو این امتیاز را رها کند
`go get` می‌گوید `requires go >= 1.26`	Go دانشجو قدیمی است: Go را به‌روز کند، یا نسخه‌های قدیمی‌تر را pin کند (در README هر نسخه آمده است)
`go: -race requires cgo`	لینوکس یا WSL بدون کامپایلر C: `sudo apt install -y build-essential` . اگر روز تصحیح فقط همین مشکل بود، نتیجه‌ی `go test -race` در CI گیت‌هاب را ملاک بگیرید

۶. تصحیح

۶.۱ روند تصحیح هر دانشجو (۱۲ تا ۱۸ دقیقه)

۰. مطمئن شوید چیزی بعد از ۳:۰۰ عوض نشده. در ریپازیتوری دانشجو `git status --porcelain` باید خالی باشد و `git rev-parse HEAD` باید با SHA ثبت‌شده در ۳:۰۰ یکی باشد. اگر نبود: `git stash -u && git checkout <sha>` و ایمیج را دوباره build و load کنید (یا فقط امتیازهایی را بدهید که روی همان کامیت قابل دفاع است).

۱. برنامه را آماده کنید. دانشجو برنامه‌اش را در Kubernetes با port-forward روی `localhost:8080` در دسترس می‌گذارد. اگر Kubernetes کار نکرد، از Compose استفاده کنید. اگر آن هم کار نکرد، برنامه را مستقیم با `go run` و `STORE=postgres` (در B: `STORE=redis`) روی یک datastore واقعی اجرا کنید، و اگر این هم نشد با `STORE=memory` .

۲. مطمئن شوید برنامه روی datastore واقعی است. امتیاز CON-1 و CON-2 (۳ امتیاز) و در نسخه‌ی A امتیاز B4 فقط روی datastore واقعی معتبر است؛ با یک Mutex درون‌حافظه‌ای این‌ها بی‌زحمت پاس می‌شوند، و port-forward فقط به **یک** pod وصل می‌شود، پس تصحیح‌گر `STORE=memory` با دو رپلیکا را از روی API تشخیص نمی‌دهد. بررسی سریع:

```
kubectl -n <ns> get cm -o yaml | grep -i STORE
kubectl -n <ns> logs deploy/<app> --tail=20
```

در Compose: `docker compose config | grep STORE` . تصحیح‌گر با `--namespace` هم یک ردیف `K8S-STORE` چاپ می‌کند و اگر `STORE=memory` باشد هشدار می‌دهد. **اگر `STORE=memory` بود، CON-1، CON-2 و (در A) B4 را صفر بگذارید.**

۳. تصحیح‌گر را اجرا کنید (حدود ۲ دقیقه). تصحیح‌گرها را از فلش به یک پوشه‌ی **بیرون از ریپازیتوری دانشجو** کپی کنید تا داخل ریپازیتوری او نروند:

```
mkdir -p ~/exam-grader
cp /path/to/usb/graders/grade_*.py ~/exam-grader/
cd /path/to/student-repo
python3 --version
docker images
```

نسخه‌ی Python باید دست‌کم 3.9 باشد. از خروجی `docker images` تگ واقعی ایمیج را برای `--image` بردارید. بعد:

```
python3 ~/exam-grader/grade_minibank.py --url http://localhost:8080 --repo . --namespace minibank
--image minibank:v1 --json ~/exam-grader/report-<name>.json
```

برای نسخه‌ی B:

```
python3 ~/exam-grader/grade_snaplink.py --url http://localhost:8080 --repo . --namespace snaplink
--image snaplink:v1 --json ~/exam-grader/report-<name>.json
```

اگر تصحیح‌گر با کد خروج `2` تمام شد، یعنی برنامه روی آدرس داده‌شده جواب نمی‌دهد: port-forward را دوباره راه بیندازید یا آدرس را درست کنید. در WSL اگر فلش دیده نمی‌شود: `sudo mkdir -p /mnt/e && sudo mount -t drvfs E: /mnt/e` .

۴. بخش‌های MANUAL را با چک‌لیست بخش ۶.۳ بررسی کنید (حدود ۳ دقیقه).

۵. دفاع حضوری (۷ تا ۱۲ دقیقه)، بخش ۶.۴.

۶. برگه‌ی نمره (بخش ۶.۶) را پر کنید.

> ⚠️ تصحیح‌گر درخواست‌های زیادی به برنامه می‌فرستد و داده‌ی آزمایشی می‌سازد. این کار روی دیتابیس دانشجو اشکالی ندارد؛ تصحیح‌گر فرض نمی‌کند دیتابیس خالی است. با `--repo` ، تصحیح‌گر `go test` و `scripts/smoke.sh` خود دانشجو را هم اجرا می‌کند.

۶.۲ خواندن گزارش تصحیح‌گر

برچسب	معنی	کار شما
AUTO	قطعی؛ تصحیح‌گر خودش بررسی کرده (قرارداد API: ۸، هم‌روندی: ۳، امتیاز اضافه‌ی B1 و B4: ۸)	همان را ثبت کنید، به شرط این‌که برنامه روی datastore واقعی بوده باشد (بند ۲ بالا)
ASSIST	پیشنهادی؛ بر اساس الگوهای متنی در فایل‌ها، `kubectl` ، `docker` و اجرای دستورها	با یک نگاه سریع تأیید یا اصلاح کنید
MANUAL	فقط انسان می‌تواند قضاوت کند	با چک‌لیست ۶.۳ نمره بدهید

هر بررسی شکست‌خورده یک دلیل یک‌خطی (انتظار در برابر واقعیت) دارد که می‌توانید همان را به دانشجو نشان بدهید. در بررسی‌های هم‌روندی، اشتباه status code جریمه‌ی دوباره ندارد (فقط یک یادداشت می‌گیرد)، چون status codeها در API-0x نمره گرفته‌اند.

۶.۳ چک‌لیست بررسی دستی (با دستورهای سریع)

مرحله	چه چیزی را ببینید	دستور سریع
S0 (۲)	«تصمیم‌های من» مشخص و واقعی است، و تا ۰:۱۵ روی GitHub push شده (SHA ثبت‌شده در ۰:۱۵)	`git show <sha-0:15>:PLAN.md | sed -n '/My decisions/,/بازرسی ۰/p'` نقطه‌ی
S0 (۱)	`go.mod` و `.gitignore` درست، و ریپازیتوری روی GitHub	`cat .gitignore; git remote -v; git status -sb`
S0 (۲)	دست‌کم ۴ merge مرحله‌ای با کامیت merge	`git log --graph --oneline | head -40`
S1 دامنه (۳)	A: نبود `float64` برای پول، `utf8.RuneCountInString`، ستون‌های `BIGINT`. B: `crypto/rand` و بررسی scheme	A: `grep -rn "float64|RuneCount|SERIAL|BIGINT" internal/` · B: `grep -rn "math/rand|crypto/rand" internal/`
S1 store (۴)	اینترفیس، دو پیاده‌سازی، انتخاب با `STORE`	`grep -rn "interface" internal/store`
S1 datastore (۳)	A: کوئری پارامتری و تراکنش (و `UPDATE` شرطی یا `FOR UPDATE`). B: `HINCRBY` و `HSETNX` یا `NX` یا Lua	A: `grep -rn "Sprintf|Begin|FOR UPDATE" internal/store` · B: `grep -rni "hincrby|setnx|nx|eval" internal/store`
S1 config (۱)	env با پیش‌فرض؛ پیکربندی نامعتبر فوراً با پیام واضح خارج شود	A: `STORE=postgres DATABASE_URL= go run ./cmd/minibank; echo $?` · B: `STORE=bogus go run ./cmd/snaplink; echo $?` (کد خروج غیرصفر و یک خط لاگ واضح)
S1 middleware (۱)	برای هر درخواست یک خط لاگ JSON با `method`، `path`، `status` و `duration_ms`، و recovery از panic	`kubectl -n <ns> logs deploy/<app> --tail=5` و `grep -rn "recover()" internal/`
S1 health (۱)	`/readyz` وقتی datastore قطع است `503` می‌دهد و `/healthz` همچنان `200`	A: `kubectl -n minibank scale deploy/postgres --replicas=0` · B: `kubectl -n snaplink scale deploy/redis --replicas=0`؛ بعد `curl -i localhost:8080/readyz`. **این را آخر کار و بعد از تصحیح‌گر انجام دهید** (بخش D3 را ببینید)
S1 shutdown (۱)	پیام خاموشی مرتب در لاگ	در یک ترمینال `kubectl -n <ns> logs -f <pod>` و در ترمینال دیگر `kubectl -n <ns> delete pod <pod>`؛ یا `docker compose stop app && docker compose logs app | tail`
S4 (۱)	`docker build` و `docker run -e` `STORE=memory` و `/healthz` جواب `200`	`docker run --rm -d -p 18080:8080 -e STORE=memory <app>:v1` و `curl -i localhost:18080/healthz`
S5 (۱)	smoke test روی Compose از clone تمیز، و ماندگاری داده	`docker compose up -d --build` بعد port-forward اول را با Ctrl+C ببندید. `scripts/smoke.sh`؛ لیست را در `/tmp/before.json` بگیرید، `docker compose down && docker compose up -d` (بدون `-v`)، و لیست را با `diff` مقایسه کنید. در پایان `docker compose down` و port-forward را دوباره راه بیندازید

مرحله	چه چیزی را ببینید	دستور سریع
S6 (۲)	CI سبز روی main و badge، و یک PR که با «Create a merge commit» ادغام شده و چک CI دارد	صفحه‌ی Actions و Pull requests در GitHub
S7 (۱)	ترتیب فایل‌ها و `kubectl apply -f k8s/` در یک مرحله	`ls k8s/` (فایل Namespace اول، مثلاً `00-namespace.yaml`) و `kubectl apply -f k8s/ --dry-run=client` بدون خطا
S8 (۲)	README کامل؛ بخش AI usage صادقانه و با یک اشتباه واقعی AI	خواندن README
B2 (۳)	درخواست واقعاً از ingress-nginx رد شود، نه فقط وجود شیء Ingress	`kubectl -n ingress-nginx port-forward svc/ingress-nginx-controller 8081:80` و `curl -i -H 'Host: minibank.local' localhost:8081/healthz` (B: `snaplink.local`) باید `200` بدهد. تصحیح‌گر برای وجود شیء Ingress فقط ۱ پیشنهاد می‌دهد؛ ۲ امتیاز بقیه با همین `curl` است
B3 (۴)	چارت خود دانشجو (در `charts/<app>/`) کل سیستم را با Helm نصب کرده	`helm list -A` (معمولاً در namespace `<app>-helm`) و `kubectl -n <app>-helm get pods`

۶.۴ دفاع حضوری (۲۰ امتیاز)

قوانین دفاع: ۷ تا ۱۲ دقیقه، روی لپ‌تاپ خود دانشجو، صفحه‌اش جلوی شما، **بدون هیچ ابزار هوش مصنوعی** (Copilot و افزونه‌های مشابه را خاموش کند). مستندات رسمی مجاز است.

مقیاس هر بخش: ۵ = واضح و درست · ۳ = نسبی یا با راهنمایی · ۱ = مبهم · ۰ = نمی‌داند

D1 — مسیر یک درخواست (۵)

از دانشجو بخواهید در کد خودش، از router تا datastore، یک درخواست را دنبال کند.

نسخه	درخواست پیشنهادی	نکته‌هایی که باید بگوید
A	`POST /transfers`	الگوی ServeMux، بعد decode با `MaxBytesReader` ، بعد اعتبارسنجی (`400` ، از جمله `from == to`)، بعد `store.Transfer(ctx)` و `BEGIN` . بعد یکی از دو راه درست: (الف) مثل راه‌حل مرجع، قفل هر دو ردیف با `SELECT … FOR UPDATE` به ترتیب id، اول نبود هر کدام (`404`)، بعد موجودی (`409`)، بعد کم کردن و اضافه کردن؛ یا (ب) `UPDATE` شرطی روی مبدأ و اضافه به مقصد، و اگر ردیفی تغییر نکرد، اول وجود هر دو حساب (`404`) و بعد موجودی (`409`). در هر دو: ثبت ردیف تراکنش، `COMMIT` ، و در آخر `201`
A	`POST /accounts/{id}/withdraw`	`r.PathValue` و `ParseInt` ، بعد `UPDATE … WHERE balance >= $1` ، بعد تشخیص «نبود حساب» از «موجودی ناکافی» با یک `SELECT`
B	`GET /{code}`	الگوی `GET /{code}` و این‌که چرا `GET /healthz` را نمی‌بلعد، بعد `store.Hit` ، بعد افزایش اتمیک که لینک حذف‌شده را زنده نکند (Lua، یا بررسی وجود و بعد `HINCRBY`)، و در آخر `302` با `Location`
B	`POST /api/links` با کد دلخواه	اعتبارسنجی URL و کد، بعد گرفتن اتمیک کد (`HSETNX` ، `SET … NX` ، یا یک اسکریپت Lua مثل راه‌حل مرجع که `EXISTS` ، `HSET` و `ZADD` را یک‌جا انجام می‌دهد)، `409` در صورت تکرار، `ZADD` برای لیست، و در آخر `201`

D2 — سؤال «چرا؟» (۵) — یکی را بپرسید

نسخه‌ی A:

سؤال	نکته‌های جواب درست
چرا پول `int64` است و نه `float64` ؟	float دودویی است و ۰.۱ را دقیق نشان نمی‌دهد؛ خطای گرد کردن جمع می‌شود؛ پول باید دقیق باشد
چرا انتقال باید داخل یک تراکنش SQL باشد؟	اتمیک بودن: اگر بین کم کردن و اضافه کردن خطا یا crash رخ دهد، پول نابود یا ساخته می‌شود
چرا `UPDATE … WHERE balance >= $1` (یا `SELECT … FOR UPDATE`) از «اول SELECT بعد UPDATE» امن‌تر است؟	بررسی و نوشتن زیر قفل ردیف انجام می‌شوند؛ دو درخواست نمی‌توانند هر دو موجودی قدیمی را ببینند
با `STORE=memory` و دو رپلیکا چه می‌شود؟	هر pod حافظه‌ی خودش را دارد؛ Service درخواست‌ها را پخش می‌کند؛ حساب‌ها گاهی پیدا نمی‌شوند و موجودی‌ها از هم جدا می‌شوند
PVC یا emptyDir برای Postgres؟	emptyDir با حذف یا جابه‌جایی pod پاک می‌شود؛ PVC می‌ماند
اگر Postgres قطع شود، liveness و readiness چه می‌کنند؟	readiness شکست می‌خورد و pod از endpointهای Service خارج می‌شود ولی restart نمی‌شود؛ liveness سالم می‌ماند؛ با برگشتن دیتابیس دوباره Ready می‌شود

نسخه‌ی B:

سؤال	نکته‌های جواب درست
چرا `HINCRBY` و نه GET و بعد SET؟	افزایش اتمیک در خود Redis؛ با GET/SET دو درخواست هم‌زمان یک مقدار را می‌خوانند و یک کلیک گم می‌شود
چرا `crypto/rand` و نه `math/rand` ؟	غیرقابل پیش‌بینی بودن؛ با `math/rand` می‌شود کدها را حدس زد و لینک‌های دیگران را پیدا کرد
چرا `302` و نه `301` ؟	مرورگر `301` را دائمی cache می‌کند؛ کلیک‌های بعدی به سرور نمی‌رسند و مقصد را هم نمی‌شود عوض کرد
چرا `javascript:` را رد می‌کنیم؟	XSS و سوءاستفاده‌ی open redirect و فیشینگ
`HSETNX` در برابر «اول EXISTS بعد SET»؟	بین بررسی و نوشتن، درخواست دیگری می‌تواند همان کد را بگیرد (race)؛ `HSETNX` (که فیلد را فقط اگر نباشد می‌نویسد) اتمیک است
با `STORE=memory` و دو رپلیکا چه می‌شود؟	لینکی که روی یک pod ساخته شده روی pod دیگر `404` می‌دهد

D3 — خرابی عمدی (۵)

یک مورد را خراب کنید و حدود ۳ دقیقه وقت بدهید. **انتظار:** دانشجو با `get pods` ، `describe` ، `logs` و `get events` علت را پیدا کند و درستش کند.

قبل از خراب کردن، نام‌ها را ببینید: `kubectl -n <ns> get deploy,sts` و نام کانتینر با `kubectl -n <ns> get deploy <app> -o jsonpath='{.spec.template.spec.containers[0].name}'`. اگر datastore یک StatefulSet است، به‌جای

`deploy/` از `sts/<name>` استفاده کنید. استقرار S7 در namespaceهای `minibank` و `snaplink` است؛ نصب Helm دانشجو (B3) در namespace جدای `<app>-helm` است و نام‌هایش فرق دارد (مثلاً `minibank-postgres`).

خرابی	دستور خراب کردن	نشانه	راه درست کردن
datastore خاموش	A: `kubectl -n minibank scale deploy/postgres --replicas=0` · B: `kubectl -n snaplink scale deploy/redis --replicas=0`	podهای اپ `0/1 READY` ، `readyz` جواب `503`	scale به ۱ (نکته‌ی زیر جدول را ببینید)
رشته‌ی اتصال یا آدرس اشتباه	A: `kubectl -n minibank set env deploy/minibank DATABASE_URL='postgres://x:y@nowhere:5432/db'` · B: `kubectl -n snaplink set env deploy/snaplink REDIS_ADDR=nowhere:6379`	pod جدید Ready نمی‌شود (یا CrashLoop) و podهای قدیمی سرویس می‌دهند	`kubectl -n <ns> rollout undo deploy/<app>`، یا حذف متغیر اضافه‌شده: A: `kubectl -n minibank set env deploy/minibank DATABASE_URL-` و بعد `kubectl apply -f k8s/` ؛ B: `kubectl -n snaplink set env deploy/snaplink REDIS_ADDR-`. فقط اصلاح ConfigMap یا `kubectl apply` این خرابی را درست نمی‌کند، چون متغیر صریح بر `envFrom` غلبه می‌کند
مسیر readiness اشتباه	`kubectl -n <ns> patch deploy <app> --type=json -p='[{"op":"replace","path":"/spec/template/spec/containers/0/readinessProbe/httpGet/path","value":"/ready"}]'`	pod جدید `0/1` می‌ماند و podهای قدیمی سرویس می‌دهند؛ در `describe` : `HTTP probe failed with statuscode: 404`	اصلاح مسیر یا `rollout undo`
تگ ایمیج اشتباه	`kubectl -n <ns> set image deploy/<app> <container>=<app>:v9`	`ErrImagePull` / `ImagePullBackOff`	`rollout undo` یا برگرداندن تگ

- همه‌ی این خرابی‌ها روی کلاستر kind با راه‌حل‌های مرجع امتحان شده‌اند. بعد از `rollout undo` ، کامل‌ترین راه برگرداندن وضعیت این است که دانشجو دوباره `kubectl apply -f k8s/` بزند (kubectl درباره‌ی annotation به نام last-applied هشدار می‌دهد).
- ⚠️ **scale به صفر با emptyDir همه‌ی داده را پاک می‌کند** (مثلاً Redis در راه‌حل مرجع B). این خرابی و بررسی S1 health را فقط بعد از اجرای تصحیح‌گر انجام دهید. در A با emptyDir، بعد از scale به ۱ باید `kubectl -n minibank rollout restart deploy/minibank` هم زده شود تا جدول‌ها دوباره ساخته شوند. بعد از هر port-forward، را دوباره راه بیندازید.
- برای دانشجویی که فقط Compose دارد: `docker compose stop db` (یا `redis`)، یا خراب کردن یک مقدار در `.env` و `docker compose up -d` .

D4 — تغییر زنده در ۳ دقیقه (۵)

بدون AI. تغییر باید کار کند و دانشجو بتواند نشانش بدهد: برای تغییرهای کد، `go test` یا `STORE=memory go run` و `curl` کافی است؛ برای تغییرهای Kubernetes، `kubectl apply` و `kubectl rollout status` . زمان build و `kind load` جزو ۳ دقیقه حساب نمی‌شود.

نسخه	تغییر
A	حداقل مبلغ انتقال ۱۰ باشد؛ کمتر از آن `400`
A	فیلد ثابت `"currency":"IRR"` به JSON حساب اضافه شود
B	URLهایی که host آن‌ها `localhost` یا `127.0.0.1` است رد شوند (`400`)
B	طول کد تولیدی از ۶ به ۷ برسد و تست مربوط هم درست شود
هر دو	رپلیکاها ۳ شود و `kubectl rollout status` نشان داده شود
هر دو	محدودیت حافظه به `64Mi` تغییر کند، apply شود و rollout دیده شود

۶.۵ اجرای قانون «توضیح بده یا از دست بده»

همان جمله‌ای که در README و PLAN دانشجو آمده: **اگر در دفاع، بعد از دو سؤال ساده‌تر، دانشجو نتواند یک مورد امتیازی (یک چک‌باکس PLAN.md) را توضیح بدهد، امتیاز همان مورد صفر می‌شود؛ چه امتیاز ساخت باشد و چه امتیاز اضافه.**

- قانون روی **مورد** اعمال می‌شود، نه کل مرحله. مثلاً اگر دانشجو نمی‌تواند بگوید `CHECK (balance >= 0)` چه می‌کند، فقط امتیاز «درستی PostgreSQL» را کم کنید.
- دو سؤال ساده‌تر: «نشانم بده کجای کد این کار انجام می‌شود» و «اگر این خط را حذف کنیم چه می‌شود؟».
- اگر دانشجو تقریباً هیچ بخشی را نمی‌تواند توضیح بدهد، کار کاملاً AI بوده؛ امتیاز موردها را یکی‌یکی صفر کنید و دلیل را در برگه بنویسید.
- استفاده از AI به‌خودی‌خود جریمه ندارد. **پنهان کردنش** (بخش AI usage خالی یا غیرواقعی) فقط امتیاز همان مورد در S8 (۱ امتیاز) را می‌گیرد؛ اگر پنهان‌کاری در دفاع آشکار شد، طبق همین قانون عمل کنید.

۶.۶ برگه‌ی نمره

بخش	حداکثر	AUTO	ASSIST/MANUAL	نهایی
S0 شروع و تصمیم‌ها	۵	–		
S1 برنامه‌ی Go	۲۵	از ۱۱	از ۱۴	
S2 تست‌ها	۸	–		
S3 Bash	۷	–		
S4 Docker	۹	–		
S5 Compose	۷	–		
S6 CI	۶	–		
S7 Kubernetes	۱۰	–		
S8 مستندات	۳	–		
کسر «توضیح بده یا از دست بده» (ساخت)	–	–		-
جمع ساخت	**۸۰**			
D1 مسیر درخواست	۵			

بخش	حداکثر	AUTO	ASSIST/MANUAL	نهایی
D2 سؤال چرا	۵			
D3 خرابی عمدی	۵			
D4 تغییر زنده	۵			
جمع دفاع	**۲۰**			
B1 متریک	۴			
B2 Ingress	۳			
B3 Helm	۴			
B4 چالش حرفه‌ای	۴			
کسر «توضیح بده یا از دست بده» (امتیاز اضافه)	–	–		-
نمره‌ی نهایی (گرد شده)	**۱۰۰ (+۱۵)**			

نام: ..... · نسخه: A / B · ریپازیتوری: ..... · SHA در ۰:۱۵: ..... · SHA در ۳:۰۰: ..... · STORE واقعی؟ بله / خیر · رتبه: ..... · قبول / مردود

---

۷. راه‌حل‌های مرجع

پوشه‌ی `instructor/reference/` برای هر نسخه یک راه‌حل کامل دارد: همه‌ی مراحل و همه‌ی امتیازهای اضافه، همراه با تاریخچه‌ی Git مرحله‌به‌مرحله و یک `DECISIONS.md` که جواب نمونه‌ی «تصمیم‌های من» است.

آنچه واقعاً تست شد (روی همین کامپیوتر، با PostgreSQL 16 و Redis واقعی):

تست	MiniBank	SnapLink
`go test -race` بدون دیتابیس	سبز	سبز
تصحیح‌گر روی Docker Compose	۸/۸ API · هم‌روندی ۳/۳ · امتیاز اضافه‌ی خودکار ۸/۸	۸/۸ API · هم‌روندی ۳/۳ · امتیاز اضافه‌ی خودکار ۸/۸
اندازه‌ی ایمیج (`scratch`، کاربر غیر root)	۱۳.۲ مگابایت	۱۰.۲ مگابایت
Kubernetes روی kind با ۲ رپلیکا	همه Ready، صفر restart، تصحیح‌گر و smoke سبز	همه Ready، صفر restart، تصحیح‌گر و smoke سبز
نصب چارت Helm (امتیاز اضافه‌ی B3)	سبز، smoke سبز	سبز، smoke سبز
Ingress با ingress-nginx روی همان کلاستر (B2، با port-forward)	`200`	`200`
`/readyz` وقتی datastore قطع است	`503` (و `/healthz` همچنان `200`)	`503` (و `/healthz` همچنان `200`)
ماندگاری داده بعد از `docker compose down` و `up`	سبز	سبز
چهار خرابی D3	همه همان نشانه‌ی جدول را دادند	همه همان نشانه‌ی جدول را دادند
نسخه‌ی خراب‌شده (پول یا کلیک گم می‌شود)	تصحیح‌گر هم‌روندی را ۰ از ۳ داد	تصحیح‌گر هم‌روندی را ۰ از ۳ داد

فقط اجرای CI روی GitHub را خودتان باید ببینید، چون به ریپازیتوری روی GitHub نیاز دارد.

کاربردها:

- قبل از آزمون، خودتان یک بار آن را روی kind اجرا کنید تا مطمئن شوید محیط کلاس آماده است.
- تصحیح‌گر را روی آن امتحان کنید تا با خروجی‌اش آشنا شوید.
- **بعد از آزمون** (و نه قبل از آن) می‌توانید آن را به‌عنوان «جواب نمونه» به دانشجوها نشان بدهید.

اجرای سریع بدون دیتابیس:

```
cd instructor/reference/minibank
STORE=memory go run ./cmd/minibank
```

اجرای کامل با Compose:

```
cd instructor/reference/minibank
docker compose up -d --build
scripts/smoke.sh
```

مراحل اجرا روی kind در README هر راه‌حل مرجع آمده است، ولی آنجا نام کلاستر `minibank` (A) و `kind` (B) است و برای Helm روش دیگری (پاک کردن namespace) آمده؛ برای تست محیط کلاس همه‌جا `--name exam` بگذارید و Helm را مثل PLAN دانشجو در namespace جدای `<app>-helm` نصب کنید.

---

۸. عدالت و جلوگیری از تقلب

- **همسایه‌ها نسخه‌ی متفاوت داشته باشند** (A و B یکی‌درمیان).
- **فقط کامیت pushشده تا ۳:۰۰ حساب است:** SHAهای ثبت‌شده با `git ls-remote` در ۰:۱۵ و ۳:۰۰ مرجع هستند، نه ساعت کامیت‌ها (که می‌شود جعلش کرد).
- **خط زمانی کامیت‌ها را ببینید:** کار واقعی در طول ۳ ساعت پخش می‌شود. یک یا دو کامیت بزرگ در دقیقه‌های آخر یعنی دفاع را عمیق‌تر کنید.

```
git log --reverse --format='%h %ad %s' --date=format:%H:%M
```

- **شباهت بین دانشجوها:** خروجی‌های AI شبیه هم می‌شوند، پس شباهت کد به‌تنهایی دلیل تقلب نیست. معیار اصلی دفاع است.
- **کمک انسانی** (پیام‌رسان، هم‌کلاسی) تنها چیز ممنوع است. اگر دیدید، طبق قانون کلاس رفتار کنید.

---

۹. رفع اشکال‌های رایج

نشانه	علت محتمل	راه حل
`ErrImagePull` یا `ImagePullBackOff`	ایمیج در کلاستر load نشده، یا تگ `latest` باعث pull از Docker Hub شده	`kind load docker-image <app>:v1 --name exam` و `imagePullPolicy: IfNotPresent`
`CrashLoopBackOff` در اپ	env اشتباه، یا اپ قبل از HTTP منتظر دیتابیس است و liveness آن را می‌کشد	`kubectl logs --previous`؛ اصلاح env؛ `startupProbe` یا بالا آوردن HTTP قبل از اتصال به دیتابیس
`Pending`	منابع کافی نیست (requests خیلی بالاست)	`kubectl describe pod` و کم کردن requests
`0/1 READY` ولی Running	`/readyz` جواب `503` می‌دهد (datastore در دسترس نیست)	بررسی Service و نام میزبان datastore و رمز
`port-forward` قطع می‌شود	pod در حال restart است	اول pod را پایدار کنید
`bind: address already in use` در port-forward	اپ Compose هنوز پورت 8080 را گرفته	`docker compose down` (بدون `-v`)
`helm install` می‌گوید `invalid ownership metadata`	چارت در همان namespace منابع `kubectl apply` نصب شده	نصب در namespace جدا: `helm install <app> charts/<app> -n <app>-helm --create-namespace`
pod کنترلر ingress-nginx روی `Pending`	مانیفست kind فقط روی نودی با برچسب `ingress-ready=true` اجرا می‌شود	`kubectl label node exam-control-plane ingress-ready=true`
`/usr/bin/env: 'bash\r'`	پایان خط ویندوزی (CRLF) در اسکریپت	`git config core.autocrlf input` و `sed -i 's/\r$//' scripts/*.sh`
`permission denied` برای اسکریپت	بیت اجرایی ثبت نشده	`chmod +x` و `git update-index --chmod=+x`
در Compose اپ به دیتابیس وصل نمی‌شود	استفاده از `localhost` به‌جای نام سرویس	`db:5432` یا `redis:6379`
`GET /accounts/999999999999` جواب `500` می‌دهد	ستون id از نوع `SERIAL` / `INTEGER` است	ستون‌ها `BIGINT` شوند (در README نسخه‌ی A آمده است)
در CI بررسی gofmt شکست می‌خورد	کد فرمت نشده	`gofmt -w .`

۱۰. شخصی‌سازی و ساخت دوباره‌ی PDFها

همه‌ی PDFها از همان فایل‌های `.md` ساخته شده‌اند. برای تغییر زمان، نمره‌ی قبولی یا هر متن دیگر:

۱. فایل `.md` مربوط را ویرایش کنید (مثلاً `version-A-minibank/README.md`).

۲. اسکریپت ساخت را اجرا کنید (`npm install` فقط بار اول لازم است):

```
cd instructor/tools
npm install
node build-pdfs.mjs
```

PDFهای دانشجو در پوشه‌ی `PDF/` و راهنمای مدرس در پوشه‌ی `instructor/` ساخته می‌شوند. اسکریپت به Node.js نسخه‌ی ۱۸ به بعد و Google Chrome نیاز دارد (اگر Chrome در مسیر استاندارد نیست: `CHROME_PATH=/path/to/chrome node build-pdfs.mjs`).

موفق باشید! 🎓